Operating Systems & Data Structures

Practical Programs in C — All 12 Codes

Operating Systems: Programs 1–6

Data Structures: Programs 7–12

Each program is provided separately with its own heading. Thread programs require a POSIX/Linux environment and the pthread compile option. The child-process program uses fork(), available on Unix/Linux.

Operating Systems — 1. Create a thread and display a message

```
#include <stdio.h>
#include <pthread.h>

void *display(void *arg)
{
    printf("Hello from thread!\n");
    return NULL;
}

int main()
{
    pthread_t t;

    pthread_create(&t, NULL, display, NULL);
    pthread_join(t, NULL);

    printf("Main thread executed.\n");
    return 0;
}
```

Compile on Linux: gcc program.c -o program -pthread

Operating Systems — 2. Fibonacci series up to n terms

```
#include <stdio.h>

int main()
{
    int n, a = 0, b = 1, c, i;

    printf("Enter number of terms: ");
    scanf("%d", &n);

    for(i = 1; i <= n; i++)
    {
        printf("%d ", a);
        c = a + b;
        a = b;
        b = c;
    }

    return 0;
}
```

Operating Systems — 3. Execute two threads with a delay

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

void *thread1(void *arg)
{
    int i;
    for(i = 1; i <= 5; i++)
    {
        printf("Thread 1: %d\n", i);
        sleep(1);
    }
    return NULL;
}

void *thread2(void *arg)
{
    int i;
    for(i = 1; i <= 5; i++)
    {
        printf("Thread 2: %d\n", i);
        sleep(1);
    }
    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```

Compile on Linux: gcc program.c -o program -pthread

Operating Systems — 4. Display processes in order of arrival (FCFS)

```
#include <stdio.h>

int main()
{
    int n, i, p[20], at[20], j, temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("Enter process ID and arrival time: ");
        scanf("%d%d", &p[i], &at[i]);
    }

    for(i = 0; i < n - 1; i++)
    {
        for(j = 0; j < n - i - 1; j++)
        {
            if(at[j] > at[j + 1])
            {
                temp = at[j];
                at[j] = at[j + 1];
                at[j + 1] = temp;

                temp = p[j];
                p[j] = p[j + 1];
                p[j + 1] = temp;
            }
        }
    }

    printf("Process execution order:\n");
    for(i = 0; i < n; i++)
        printf("P%d ", p[i]);

    return 0;
}
```

Operating Systems — 5. Display numbers using a loop

```
#include <stdio.h>

int main()
{
    int i;

    for(i = 1; i <= 10; i++)
    {
        printf("%d ", i);
    }

    return 0;
}
```

Operating Systems — 6. Create a child process and display a message

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Hello from child process!\n");
    }
    else if(pid > 0)
    {
        wait(NULL);
        printf("Hello from parent process!\n");
    }
    else
    {
        printf("Process creation failed.\n");
    }

    return 0;
}
```

Requires Unix/Linux because fork() is not available in standard Windows C.

Data Structures — 7. Student details using a structure

```
#include <stdio.h>

struct Student
{
    int roll;
    char name[50];
    float marks;
};

int main()
{
    struct Student s;

    printf("Enter roll number: ");
    scanf("%d", &s.roll);

    printf("Enter name: ");
    scanf("%49s", s.name);

    printf("Enter marks: ");
    scanf("%f", &s.marks);

    printf("\nStudent Details\n");
    printf("Roll Number: %d\n", s.roll);
    printf("Name: %s\n", s.name);
    printf("Marks: %.2f\n", s.marks);

    return 0;
}
```

Data Structures — 8. Create and display a singly linked list

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

int main()
{
    struct Node *head = NULL, *temp, *newNode;
    int n, i, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        newNode = malloc(sizeof(struct Node));

        if(newNode == NULL)
            return 1;

        printf("Enter data: ");
        scanf("%d", &value);

        newNode->data = value;
        newNode->next = NULL;

        if(head == NULL)
            head = newNode;
        else
            temp->next = newNode;

        temp = newNode;
    }

    printf("Linked List: ");
    temp = head;

    while(temp != NULL)
    {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }

    printf("NULL");

    while(head != NULL)
    {
        temp = head;
        head = head->next;
        free(temp);
    }

    return 0;
}
```

Data Structures — 9. Stack push and pop operations

```
#include <stdio.h>
#define MAX 5

int stack[MAX], top = -1;

void push(int value)
{
    if(top == MAX - 1)
        printf("Stack Overflow\n");
    else
        stack[++top] = value;
}

void pop()
{
    if(top == -1)
        printf("Stack Underflow\n");
    else
        printf("Popped element: %d\n", stack[top--]);
}

int main()
{
    push(10);
    push(20);
    push(30);

    printf("Stack elements: ");
    for(int i = top; i >= 0; i--)
        printf("%d ", stack[i]);

    printf("\n");
    pop();

    return 0;
}
```

Data Structures — 10. Queue enqueue and dequeue operations

```
#include <stdio.h>
#define MAX 5

int queue[MAX], front = 0, rear = -1;

void enqueue(int value)
{
    if(rear == MAX - 1)
        printf("Queue Overflow\n");
    else
        queue[++rear] = value;
}

void dequeue()
{
    if(front > rear)
        printf("Queue Underflow\n");
    else
        printf("Deleted element: %d\n", queue[front++]);
}

int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);

    printf("Queue elements: ");
    for(int i = front; i <= rear; i++)
        printf("%d ", queue[i]);

    printf("\n");
    dequeue();

    return 0;
}
```

Data Structures — 11. Create a binary search tree and display elements

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *left, *right;
};

struct Node *insert(struct Node *root, int value)
{
    if(root == NULL)
    {
        root = malloc(sizeof(struct Node));
        if(root == NULL)
            exit(1);

        root->data = value;
        root->left = root->right = NULL;
    }
    else if(value < root->data)
        root->left = insert(root->left, value);
    else if(value > root->data)
        root->right = insert(root->right, value);

    return root;
}

void inorder(struct Node *root)
{
    if(root != NULL)
    {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

void freeTree(struct Node *root)
{
    if(root != NULL)
    {
        freeTree(root->left);
        freeTree(root->right);
        free(root);
    }
}

int main()
{
    struct Node *root = NULL;
    int n, i, value;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("Enter element: ");
        scanf("%d", &value);
        root = insert(root, value);
    }

    printf("BST elements: ");
    inorder(root);

    freeTree(root);
    return 0;
}
```

Data Structures — 12. Search for an element in an array

```
#include <stdio.h>

int main()
{
    int a[100], n, key, i, found = 0;

    printf("Enter array size: ");
    scanf("%d", &n);

    printf("Enter array elements: ");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter element to search: ");
    scanf("%d", &key);

    for(i = 0; i < n; i++)
    {
        if(a[i] == key)
        {
            printf("Element found at position %d\n", i + 1);
            found = 1;
            break;
        }
    }

    if(!found)
        printf("Element not found\n");

    return 0;
}
```